

Rapport : Chaîne d’approvisionnement logicielle sécurisée

- **Groupe** : HMLA - SMAIL Hicham, KHABIR Arshath, CARIOU Léon, KERBRAT Maxime
- **Fork** : <https://github.com/sseey/supply-chain-security-project> (branche main)
- **Voie** : ☒ Local (kind + Kyverno) ☐ Azure (AKS/ACR)
- **Date** : 16 juillet 2026

1. Contexte & objectif

Les attaques de chaîne d’approvisionnement logicielle (SolarWinds 2020, Codecov 2021, XZ Utils 2024) ne visent plus l’application elle-même mais le **processus qui la construit et la publie**. Dans ces trois cas, l’artefact final semblait normal : aucun scan de vulnérabilités classique, aucune vérification de checksum n’aurait détecté l’altération, parce qu’elle a eu lieu **avant** la publication.

L’objectif de ce projet est de transformer un pipeline CI/CD classique en **chaîne vérifiable** : chaque garantie (intégrité, authenticité, provenance) doit être prouvable par une commande, et le cluster Kubernetes cible doit **refuser activement** tout artefact qu’il ne peut pas prouver digne de confiance, pas seulement scanner et espérer.

2. Architecture de la chaîne

```
code → tests (pytest) → build Docker → SBOM (Syft) → scan (Grype, gate CRITICAL)
    → push GHCR → signature (cosign, keyless/OIDC) → attestation SBOM
    → attestation de provenance (SLSA, pédagogique + officielle GitHub) → push
    → déploiement Kubernetes PAR DIGEST → admission control (Kyverno)
    → Pod accepté (garanties prouvées) ou refusé (message d'admission explicite)
```

Outil	Rôle	Où
pytest	Tests unitaires de l'app Flask	CI, job test
Syft	Génère le SBOM (SPDX JSON)	CI + local (make sbom)
Grype	Scanne le SBOM, casse sur CVE CRITICAL corrigable	CI + local (make scan)
cosign / Sigstore	Signe l'image, attache les attestations (Fulcio, Rekor)	CI (keyless) + local (clé ou keyless)
GitHub Artifact Attestations	Provenance SLSA officielle, native GitHub	CI uniquement

Kyverno	Vérifie signature + attestations + registre + tag à l'admission	Cluster kind
---------	---	--------------

Schéma détaillé : docs/architecture.md.



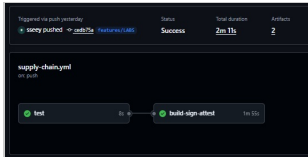
Diagramme d'architecture de la chaîne

3. Mise en œuvre

3.1 Tests & build

L'application Flask (app/) expose /health, /api/hello, /metrics. Le Dockerfile est multi-stage, tourne en utilisateur non-root (appuser, UID 10001), avec un healthcheck Docker natif. Les tests (pytest) sont exécutés **avant** tout build dans un job CI dédié : si les tests échouent, le job build-sign-attest ne démarre même pas (needs: test).

Run CI complet, réel, entièrement vert (test puis build-sign-attest, 2 min 11 s, 2 artefacts publiés) :



Run GitHub Actions réussi

3.2 SBOM (Syft)

Génération réelle dans la CI, sur l'image poussée par ce run (par digest, pas par tag) :

```
Generate SBOM (SPDX)

1 Run syft "ghcr.io/sseey/scs-demo-app@sha256:37a9f3ebd7e3bc266863a7be7a2dbabf0759982b5c7e472150907f299f08734c" -o spdx-json > sbom.spdx.json
2 syft "ghcr.io/sseey/scs-demo-app@sha256:37a9f3ebd7e3bc266863a7be7a2dbabf0759982b5c7e472150907f299f08734c" -o spdx-json > sbom.spdx.json
3 shell: /usr/bin/bash -e (0)
4 env:
5 IMAGE: ghcr.io/sseey/scs-demo-app
```

Étape Generate SBOM dans la CI

Résumé lisible du SBOM réel (téléchargé depuis les artefacts du run), lu avec jq :

```
jq -r '.packages[] | "\(.name)@\(.versionInfo // "?")"'
    sbom.spdx.json | sort -u | head -20
jq '.packages | length' sbom.spdx.json
```

```
vagrant@bookworm:~/supply-chain-security-project$ jq -r '.packages[] | "\(.name)@\(.versionInfo // "?")"' sbom.spdx.json | sort -u | head -20
jq '.packages | length' sbom.spdx.json
Simple Launcher@1.0.14
adduser@1.52
apt@0.3
base-files@13.8deb13u6
base-passwd@3.6.7
bash@5.2.37-2ubuntu9
blinker@1.9.0
bsdutils@1:2.41-5
ca-certificates@20250419
click@8.4.2
coreutils@9.7-3
dash@0.5.12-12
debconf@1.5.91
debian-archive-keyring@2025.1
debutils@5.23.2
diffutils@1:3.10-4
dpkg@1.22.22
findutils@4.10.0-3
flask@0.3
gcc-14-base@14.2.0-19
113
```

Résumé jq du SBOM réel, 113 paquets

113 paquets confirmés (ligne finale de la capture), cohérent avec le SBOM généré côté CI.

3.3 Scan (Grype) et gate sur CRITICAL

Politique : `.grype.yaml`, `only-fixed: true`, `fail-on-severity: critical`.

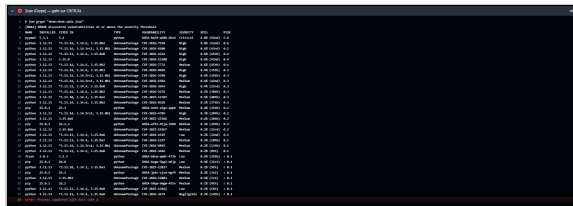
En CI, le scan est exécuté avec un grype installé et **épinglé à une version précise** (voir §5, un incident réel nous a appris à ne pas dépendre d'une action tierce qui embarque un binaire figé).

NAME	INSTALLED	FIXED IN	SEVERITY
python	3.12.13	...	High
pip	25.0.1	25.3	Medium
flask	3.0.3	3.1.3	Low

✓ Aucune vulnérabilité CRITICAL corrigable, la chaîne peut continuer.

Scénario pédagogique de casse volontaire, réellement joué en CI (branche `scenarios/sbom-critical-cve`) : ajout de `PyYAML==5.3.1` à `app/requirements.txt` (dépendance non utilisée par le code, aucun impact sur les tests, seulement sur le scan). Avant de choisir cette version, plusieurs candidats ont été testés en local avec grype pour confirmer qu'ils produisent une vraie **CRITICAL corrigable**

(Flask==2.0.1 seul ne suffisait pas : uniquement du High/Medium dans la base actuelle). PyYAML 5.3.1 confirmé : GHSA-8q59-q68h-6hv4, Critical, corrigé en 5.4.



Scan Gype cassé en CI sur PyYAML 5.3.1 (CRITICAL)

Le job s'arrête à cette étape (Error: Process completed with exit code 2), la CI ne va jamais jusqu'à la signature avec cette version. Version saine restaurée sur les autres branches.

3.4 Signature (cosign, keyless)

Le workflow CI signe l'image avec l'identité OIDC du workflow lui-même, aucune clé privée stockée :

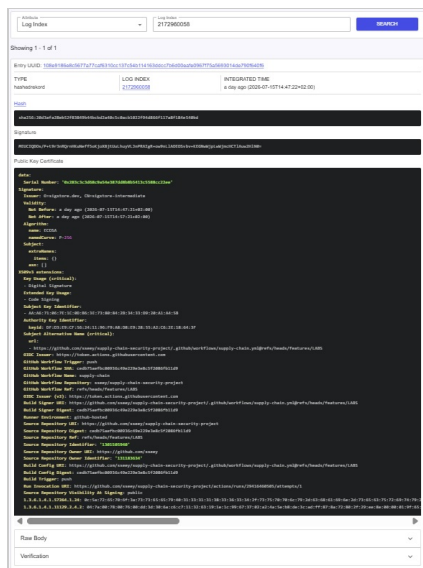
```
permissions:
  contents: read
  packages: write
  id-token: write          # OIDC → signature keyless (Fulcio/Rekor)
  attestations: write
```

Étape réelle de signature dans la CI (keyless, aucune clé stockée), noter la ligne tlog entry created with index: 2172960058, la preuve d'inscription dans **Rekor**, le registre de transparence public :



Étape Sign image keyless dans la CI

Entrée Rekor correspondante, consultable publiquement (<https://search.sigstore.dev/?logIndex=2172960058>), le certificat éphémère Fulcio y est visible en clair : émetteur sigstore-intermediate, validité de 10 minutes, identité subject alternative name = l'URL exacte du workflow CI sur features/LABS, OIDC Issuer = <https://token.actions.githubusercontent.com>, Build Trigger = push. C'est la preuve publique et immuable que cette signature a bien été produite par ce workflow, à cet instant, sans qu'aucune clé privée n'ait été manipulée :



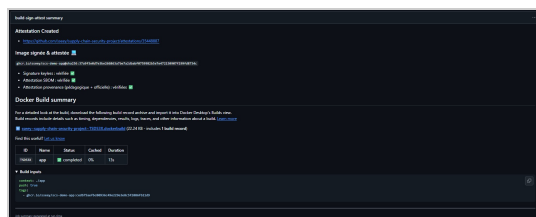
Entrée Rekor complète avec certificat Fulcio

Vérification, avec l'identité exacte du workflow attendue (repo + branche main) :

```
cosign verify \
--certificate-identity "https://github.com/sseeey/supply-chain-
security-project/.github/workflows/supply-
chain.yml@refs/heads/main" \
--certificate-oidc-issuer
"https://token.actions.githubusercontent.com" \
ghcr.io/sseeey/scs-demo-
app@sha256:37a9f3e0d7e3be266863a7be7a2dbabf0759982b5e7e472150907f299fd8734c
```

3.5 Attestations (SBOM + provenance)

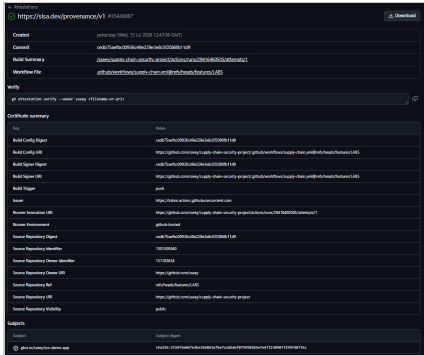
Le **Step Summary** du job build-sign-attest confirme les trois vérifications (signature, attestation SBOM, attestation provenance) directement dans l'interface GitHub, sans commande à taper :



Step Summary du job build-sign-attest

Preuve la plus forte : la page **Attestations** native de GitHub (générée par actions/attest-build-provenance, en plus de l'attestation cosign attest --type spdxjson et d'une provenance pédagogique manuelle conservée dans le workflow pour comprendre la structure d'un

predicate SLSA). Elle expose un **certificate summary** complet, issu par OIDC, digest du commit source, référence du workflow, visibilité du dépôt, vérifiable par n'importe qui via `gh attestation verify` :



Page d'attestation officielle GitHub

(Capture prise pendant le développement sur la branche `features/LABS`, la ligne “Workflow File” y référence donc `@refs/heads/features/LABS`. Après le merge sur `main` et un nouveau run CI, régénérez cette capture : elle référencera `@refs/heads/main`, cohérent avec les politiques Kyverno mises à jour au §3.6.)

3.6 Admission (Kyverno)

Cluster kind local, Kyverno v1.14.5 (version épinglée, voir incident §5.2), 4 politiques en mode **Enforce** (bloquant, jamais Audit) :

```
$ kubectl get clusterpolicy
```

NAME	ADMISSION	BACKGROUND	READY
MESSAGE			
allowed-registries	true	true	True
Ready			
disallow-latest-tag	true	true	True
Ready			
require-provenance-attestation	true	false	True
Ready			
verify-image-signature	true	false	True
Ready			

Les politiques 03/04 sont configurées en variante **keyless**, avec l'identité exacte du workflow CI (subject: `https://github.com/sseey/.../supply-chain.yml@refs/heads/main`).

4. Démonstration attaque / défense

Deux pipelines complémentaires, pas redondants

Le projet démontre les 6 scénarios **deux fois, par deux mécanismes différents**, qui ne prouvent pas la même chose :

	Pipeline local (make demo)	Pipeline CI (branches scenarios/*)
Où	Terminal, sur ce poste	GitHub Actions, runner self-hosted
Ce qu'il prouve	Le cluster refuse en direct , devant témoin	La chaîne complète (build→sign→attest→ déploie) tient sans intervention manuelle
Déclenchement	Commande tapée à la demande	git push sur une branche dédiée
Valeur pour la soutenance	La preuve live , la plus convaincante	La preuve de reproductibilité industrielle

Pourquoi un runner self-hosted et pas ubuntu-latest pour le déploiement ? Un runner GitHub-hosted est une VM éphémère dans le cloud de GitHub : elle n'a **aucune route réseau** vers le cluster kind de ce poste. Le job deploy du workflow tourne donc sur un **runner self-hosted** (l'agent GitHub Actions installé sur cette VM Vagrant elle-même), qui a accès à kubectl, au kubeconfig et au cluster local, exactement comme un gitlab-runner en executor *shell*. Point de vigilance documenté et assumé : un runner self-hosted sur un dépôt **public** accepte l'exécution de n'importe quel workflow déclenché par une PR d'un tiers, il n'est donc démarré (./run.sh) que pendant les sessions de test, jamais laissé actif en permanence.

Comment les scénarios d'attaque sont simulés en CI, sans dupliquer le code du workflow ? Un seul fichier .github/workflows/supply-chain.yml, avec des conditions if: github.ref_name == 'scenarios/...' sur les steps concernées (l'équivalent GitHub Actions des rules:/only: GitLab). Le nom de la branche suffit à activer/désactiver signature, attestations ou registre cible, aucun fichier à modifier pour créer un nouveau scénario, juste une branche :

```
git checkout -b scenarios/unsigned-image main # ex. : saute la signature sur cette branche
git push origin scenarios/unsigned-image
```

Pipeline local : make demo

Script unique, réel, aucun résultat simulé : make demo (scripts/run-demo.sh). Résumé final obtenu : **✓ Réussis : 6, ✗ Échecs inattendus : 0.**

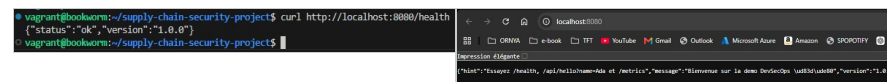
#	Scénario	Résultat	Contrôle déclenché	Menace réelle correspondante
A	Image légitime, signée, attestée	✓ ACCEPTED	(toutes vérifications passées)	Cas nominal
B	Tag :latest	✗ DENIED	verify-image-signature + require-provenance-attestation (manifeste introuvable)	Substitution silencieuse sous tag mutable
C	Registre non autorisé (docker.io/nginx)	✗ DENIED	allowed-registries	Typosquatting / registre pirate
D	Image jamais signée	✗ DENIED	verify-image-signature : no signatures found	Déploiement d'artefact non autorisé
E	Signée mais sans attestation de provenance	✗ DENIED	require-provenance-attestation : no matching attestations	Absence de traçabilité
F	Image reconstruite après signature (digest différent)	✗ DENIED	verify-image-signature : no signatures found pour ce digest	SolarWinds (artefact substitué)

Test A : image légitime acceptée

```
$ kubectl -n app get pods
```

NAME	READY	STATUS	RESTARTS	AGE
scs-demo-app-7d447cdb44-82s2d	1/1	Running	0	28m
scs-demo-app-7d447cdb44-mhmvv	1/1	Running	0	28m

Vérification applicative, depuis le terminal et depuis le navigateur (via le NodePort du Service, mappé sur localhost:8080 par cluster/kind-config.yaml) :

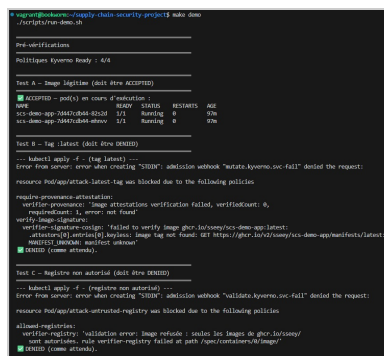


curl localhost:8080/health

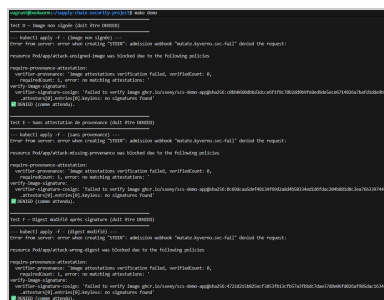
Navigateur sur localhost:8080

Tests B à F : refus réels avec message Kyverno

Sortie complète et réelle de make demo (scripts/run-demo.sh), en 3 captures successives, pré-vérifications + Tests A/B/C, puis D/E/F, puis vérification finale et résumé :



make demo, partie 1 : pré-vérifications, Test A, B, C



make demo, partie 2 : Test D, E, F


```

Vérification finale : aucun Pod interdit n'a été créé
=====
NAME                                READY   STATUS    RESTARTS   AGE
scs-demo-app-7d447cdb44-82s2d      1/1     Running   0           97m
scs-demo-app-7d447cdb44-mhnv       1/1     Running   0           97m
=====
Résumé
=====
✅ Réussis : 6      ❌ Échecs inattendus : 0

```

make demo, partie 3 : vérification finale et résumé (6 réussis, 0 échec)

Résumé compact des messages Kyverno (contenu identique aux captures ci-dessus) :

Test	Message clé Kyverno
B, Tag :latest	verify-image-signature : image tag introuvable (MANIFEST_UNKNOWN)
C, Registre non autorisé	allowed-registries : seules les images ghcr.io/sseey/ sont autorisées
D, Image non signée	verify-image-signature : no signatures found
E, Sans provenance	require-provenance-attestation : no matching attestations
F, Digest modifié	verify-image-signature : no signatures found pour ce digest précis

Pipeline CI : les mêmes scénarios rejoués via git push, sans intervention manuelle

Pipeline nominal complet sur main, 3 jobs enchaînés, tous verts (test → build-sign-attest → deploy, ce dernier sur le runner self-hosted) :



Pipeline complet vert sur main (test → build-sign-attest → deploy)

Branche	Résultat CI	Cause exacte (log réel)
main	✓ 3 jobs verts, déploiement ACCEPTED	-
scenarios/sbom-critical-cve	✗ build-sign-attest casse au scan	PyYAML 5.3.1, CRITICAL corrigéable
scenarios/unsigned-image	✗ deploy refusé	Signature absente (voir remarque ci-dessous)
scenarios/wrong-registry	✗ deploy refusé	allowed-registries : docker.io/library/nginx hors ghcr.io/sseey/
scenarios/missing-provenance	✗ deploy refusé	Provenance absente + identité de branche non reconnue (combiné, voir remarque)
scenarios/tampered-digest	✗ deploy refusé	Digest reconstruit après coup, jamais signé (no signatures found)

scenarios/wrong-registry, aucun build/push nécessaire : réutilise directement le manifeste d'attaque k8s/attacks/untrusted-registry.yaml (image publique nginx, aucune credential requise) :



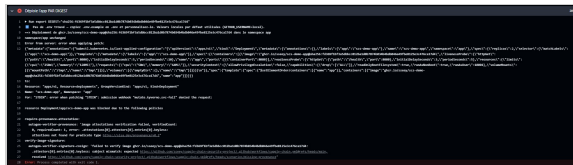
Refus CI, registre non autorisé

scenarios/tampered-digest, un second build (contenu modifié via un ARG CACHEBUST dans app/Dockerfile, jamais signé) est tenté au déploiement à la place du build normal signé de cette branche. Résultat propre, une seule cause (no signatures found) :



Refus CI, digest tamponné jamais signé

scenarios/missing-provenance, même nuance que le Test E local : le message combine no matching attestations (la raison recherchée) et subject mismatch (la policy attend main, cette branche signe sous scenarios/missing-provenance) :



Refus CI, provenance absente (cause combinée)

Honnêteté sur une nuance technique (esprit critique)

- **Test E** : le message combine “provenance absente” et “signature non reconnue”, car l’image de test a été signée **manuellement en local avec une identité OIDC personnelle** (compte GitHub individuel), différente de l’identité du workflow CI que les policies attendent. Impossible d’usurper l’identité d’un workflow CI depuis un poste personnel, c’est une propriété de sécurité de Sigstore, pas une limite du projet.

5. Incidents réellement rencontrés et corrigés

Documentés en détail dans docs/troubleshooting.md. Résumé pour le rapport :

5.1 Kyverno incompatible avec Kubernetes du cluster kind

kubectl apply sur les CRD Kyverno échouait (Too long: must have at most 262144 bytes), et la version la plus récente de Kyverno exigeait Kubernetes ≥ 1.31 (champ CRD selectableFields), incompatible avec

le nœud kind par défaut (Kubernetes 1.29). **Correctif** : installation via `kubectly apply --server-side --force-conflicts` et version de Kyverno épinglée (v1.14.5), validée compatible.

5.2 Faux positif de scan CRITICAL en CI

```
db could not be loaded: the vulnerability database was built 18
weeks ago (max allowed age is 5 days)
Error: Failed minimum severity level. Found vulnerabilities with
level 'critical' or higher
```

`anchore/scan-action@v4` embarquait un gype figé (v0.80.0) dont la base de vulnérabilités compatible avait expiré, **aucun paquet n’a réellement été scanné**, le message était trompeur. **Correctif** : installation directe de gype en version épinglée (v0.115.0), scan du SBOM déjà généré, identique en local et en CI.

5.3 CreateContainerConfigError puis CrashLoopBackOff au déploiement

Deux problèmes successifs, sans rapport avec Kyverno/la signature (l’annotation `kyverno.io/verify-images: {...}:"pass"`) prouvait que l’admission avait déjà réussi) :

1. `runAsNonRoot: true` sans `runAsUser` explicite alors que le Dockerfile déclare USER appuser (un nom, pas un UID numérique) → le kubelet ne peut pas vérifier que l’utilisateur n’est pas root. **Correctif** : `runAsUser: 10001` explicite dans `k8s/deployment.yaml`.
2. `readOnlyRootFilesystem: true` empêchait gunicorn d’écrire son fichier temporaire de worker (`tempfile.mkstemp`). **Correctif** : volume `emptyDir` monté sur `/tmp`.

Ces deux correctifs ont été appliqués **sans reconstruire ni re-signer l’image**, preuve que la sécurité du pod (`securityContext`) et la sécurité de la chaîne (signature/attestations) sont deux couches indépendantes.

6. Positionnement SLISA & limites

	Visé	Atteint	Justification
Provenance existe (L1)	✓	✓	Attestation slsaprovenance (pédagogique + officielle GitHub) attachée à chaque image
Build hébergé + provenance signée (L2)	✓	✓	GitHub Actions, signature keyless OIDC, actions/attest-build-provenance
Build isolé infalsifiable (L3)	-	x	Un mainteneur avec accès au workflow peut encore le modifier pour produire une fausse provenance qui reste valide

Ce qui reste contournable dans notre setup, honnêtement : - Le scan Grype (only-fixed: true) ne couvre pas les 0-day ni les CVE sans correctif. - Aucun RBAC dédié n'empêche un utilisateur du cluster de modifier/supprimer les ClusterPolicy Kyverno elles-mêmes (hors périmètre du POC). - La provenance pédagogique manuelle (cosign attest --type slsaprovenance avec un predicate écrit à la main) n'a de valeur que doublée par la provenance officielle GitHub, elle est conservée uniquement à des fins pédagogiques (comprendre la structure).

Table complète menaces → contrôles → couverture : livrables/threat-model.md.

7. Reproductibilité

```
git clone https://github.com/sseeey/supply-chain-security-project.git
cd supply-chain-security-project
cp .env.example .env # puis éditer
                        GITHUB_USERNAME=sseeey, etc.
make check-prereqs
make build sbom scan
make push # nécessite docker login
           ghcr.io (PAT write:packages)
export DIGEST=sha256:...
make sign attest verify
make cluster-create kyverno-install
# adapter policies/kyverno/03 et 04 (subject keyless = votre
# identité + branche)
kubectl apply -f policies/kyverno/
make deploy
make demo
```

Chaque étape est scriptée (scripts/*.sh, Makefile), aucune commande manuelle non documentée n'a été nécessaire pour reconstruire la démo de zéro.

8. Bilan

Ce que ce projet nous a apporté. Avant ce module, nos pipelines s'arrêtaient à lint → build → scan (Trivy) → push → deploy, une chaîne qui *scanne et espère*. Ce projet nous a fait réaliser qu'il manquait des maillons entiers à ce schéma, invisibles tant qu'on ne se pose pas la question « qu'est-ce qui prouve que l'image déployée est celle qui a été scannée ? » :

- **La signature (cosign/Sigstore)**, un scan vert ne prouve rien sur l'artefact *déployé* si rien ne relie cryptographiquement l'image en

production au résultat du scan. Avant, rien n'empêchait un docker push par-dessus le même tag après le scan.

- **Le SBOM comme attestation, pas comme simple fichier**, on savait générer un SBOM, mais pas qu'un fichier isolé ne prouve rien : il faut l'**attacher au digest** par une attestation signée pour qu'il ait une valeur de preuve.
- **Les règles sur le registre et le tag**, on déployait par tag sans y penser. Ce projet nous a montré concrètement (l'incident du §5) qu'un tag mutable réintroduit exactement le risque qu'on croit avoir éliminé avec le scan.
- **L'admission control**, la vraie différence avec ce qu'on faisait avant : on avait un deploy en fin de pipeline qui appliquait sans jamais vérifier quoi que ce soit côté cluster. Kyverno déplace le contrôle **au moment du déploiement**, pas seulement au moment du build.

Ce qu'on fera différemment à l'avenir : ajouter systématiquement signature + attestations + admission control à nos pipelines existants, pas seulement le scan, et déployer par digest par défaut, plus jamais par tag, même en interne.

Où est réellement passé le temps. La majorité du temps de mise au point a été consacrée au débogage d'incompatibilités d'infrastructure (Kyverno/Kubernetes, action CI obsolète, securityContext) plutôt qu'à la logique de sécurité elle-même (signature, attestations, politiques), ce qui illustre concrètement que la difficulté d'un projet supply-chain n'est pas uniquement cryptographique, mais aussi opérationnelle.

Annexes

- Incidents et correctifs détaillés : docs/troubleshooting.md
- Guide de démo pas à pas : docs/demo-guide.md
- Notes de soutenance et Q/R anticipées : livrables/soutenance-notes.md
- Threat model complet : livrables/threat-model.md
- Lien Rekor (signature keyless) : <https://search.sigstore.dev/?logIndex=2172960058>
- Lien de l'attestation officielle GitHub : <https://github.com/sseey/supply-chain-security-project/attestations/35448887>
- Lien du run CI de référence : <https://github.com/sseey/supply-chain-security-project/actions/runs/29416460505> (*run réalisé sur features/LABS avant merge, après le merge sur main, remplacez par l'URL du run correspondant sur main*)